COS30018 - Intelligent System

Project: Stock Price Prediction System

Task C.7 - Extension

Student Name: Ngo Sy Vuong
Student ID: 105551480
Tutor: Nguyen Manh Toan
Tutorial Session: Wednesday 8:00 - 12:00
Semester: May - July 2026

SWINBURNE UNIVERSITY OF TECHNOLOGY

Table of Contents

1. Introduction

This project presents an end-to-end stock directional prediction framework evaluated on five major equities from the Australian Securities Exchange (ASX): Commonwealth Bank Australia (CBA), BHP Group (BHP), National Australia Bank (NAB), CSL Limited (CSL), and Westpac Banking Corporation (WBC). Built on 5 years of daily historical OHLCV data, the system engineers 18 stationary technical indicators capturing momentum, trend, volatility, and volume dynamics. Furthermore, an automated NLP extraction pipeline gathers financial news feeds, processes them through a pre-trained FinBERT model, and applies calendar-decay propagation to construct daily sentiment signals.

The primary objective of this report is to benchmark 12 distinct model variations across two structural hyperparameter environments (config_1_standard and config_2_deep) and three core recurrent network backbones (LSTM, GRU, Vanilla RNN). Through comparative evaluation and formal statistical hypothesis testing, this study assesses whether augmenting classical technical indicators with NLP sentiment vectors provides statistically significant performance gains or introduces noise-induced mode collapse in next-day equity directional prediction.

2. Environment Setup

To successfully replicate and run the stock price prediction model, please fulfill the following structural and software dependencies.

2.1 Environment

The system is built on an isolated virtual environment (venv) to ensure dependency abstraction and eliminate version conflicts with global system libraries:

- Core Runtime: **Python 3.11.x (64-bit)**
- Execution Interface: **Windows Command Prompt (CMD)**

2.2 Deployment

To reconstruct this environment layout from scratch, run the following sequential commands within the target file directory: **(make sure you have installed Python 3.11)**

```
py -3.11 -m venv venv

venv\Scripts\activate.bat

pip install -r requirements.txt
```

Every time running a python file in this project's folder, please active the virtual environment first using **venv\Scripts\activate.bat**

2.3 System Structure

data_process.py	Run this file to extract the OHLVC features for the 5 tickers
feature_engineer.py	Run this file to extract the technical indicators (18 features and 1 target) from the OHLVC features
data_preprocess.py	Run this file to convert the engineered technical features into sequential inputs for the binary classification models
training.py	Run this file to train the baseline models (models trained without sentiment score)
news_extract.py	Run this file to gather financial headlines from GNews
sentiment_extract.py	Run this file to extract the sentiment score from the financial headlines
sentiment_merge.py	Run this file to merge the sentiment score extracted into the technical dataset
training_sentiment.py	Run this file to train the sentiment models (models trained with sentiment score)
evaluation.py	Run this file to produce data to support evaluation
hypothesis_testing.py	Run this file to produce data to support hypothesis testing

3. Data Processing

The process of Data Processing happens inside the data_process.py file.

The data processing stage is responsible for collecting and preparing historical stock market data before feature engineering and model training. The system uses the yfinance library to retrieve historical data for five Australian Securities Exchange (ASX) stocks: CBA, BHP, NAB, CSL, and WBC. For each stock, five years of daily historical data are collected.

The downloaded datasets contain the standard OHLCV features: Open, High, Low, Close, and Volume. These features provide the fundamental market information required for subsequent technical indicator calculation and model training.

After downloading the data, several preprocessing steps are performed. First, MultiIndex columns generated by yfinance are flattened and column names are standardized. The system then checks that all required OHLCV columns are available and removes any unnecessary columns. The date index is converted into a standard datetime format, named Date, and sorted chronologically to maintain the correct temporal order of the observations.

Missing and invalid data are also handled during this stage. Rows containing entirely missing values are removed, while remaining missing values are filled using forward-filling followed by backward-filling. Observations with invalid closing prices or negative trading volumes are removed to ensure that the dataset contains valid market information.

Finally, each processed dataset is saved as an individual CSV file in a local data_cache directory. The processed DataFrames are also stored in a dictionary using their ticker symbols as keys, allowing them to be easily accessed during the subsequent feature engineering stage. This process ensures that all stock datasets follow a consistent structure and are ready for calculating technical indicators and training the prediction models.

4. Feature Engineering

The feature engineering process is implemented in feature_engineering.py. It transforms the cleaned OHLCV datasets into 18 stationary and scale-invariant technical features. Raw price levels are converted into relative measures of momentum, trend, volatility, and trading activity, making the features more comparable across different stocks.

4.1 Price Momentum and Returns (3 features)

1. 1-day return (return_1d) measures short-term daily price momentum.
2. 5-day return (return_5d) captures price momentum over approximately one trading week.
3. 20-day return (return_20d) captures longer-term momentum over approximately one trading month.

4.2 Intraday Candlestick Dynamics (2 features)

4. High-Low ratio (high_low_ratio) measures the daily trading range relative to the closing price, providing an indication of intraday volatility.
5. Close-Open ratio (close_open_ratio) measures the percentage movement between the opening and closing prices, indicating the direction and magnitude of daily price movement.

A small constant 10^−8 is added to denominators to prevent division-by-zero errors.

4.3 Moving Average Distance (3 features)

6. 20-day SMA distance (dist_sma_20) measures the current price relative to its short-term moving average.
7. 50-day SMA distance (dist_sma_50) measures the current price relative to its medium-term moving average.
8. 200-day SMA distance (dist_sma_200) measures the current price relative to its long-term moving average.

These features help the models identify whether a stock is trading above or below its short-, medium-, and long-term trends.

4.4 Relative Strength Index (1 feature)

9. RSI (rsi_14) is a 14-day momentum indicator ranging approximately from 0 to 100. It provides information about the strength of recent upward or downward price movements.

4.5 MACD Features (3 features)

The MACD is calculated using 12-day and 26-day exponential moving averages and normalized by the closing price.

10. MACD line (macd_line) represents the difference between short- and long-term momentum.
11. MACD signal (macd_signal) is a 9-period EMA of the MACD line that provides a smoothed momentum signal.
12. MACD histogram (macd_hist) represents the difference between the MACD line and signal line, highlighting momentum changes.

4.6 Volatility Features (3 features)

13. Bollinger Band width (bb_width) measures the relative width of the Bollinger Bands and indicates market volatility.
14. Bollinger Band percentage (bb_pct) indicates the current price position within the Bollinger Band range.
15. Normalized ATR (norm_atr) measures recent price volatility using Average True Range, normalized by the closing price for comparability between stocks.

4.7 Volume and Liquidity Features (3 features)

16. Volume ratio (volume_ratio) compares current trading volume with its 20-day average.
17. 1-day volume change (volume_change_1d) measures the daily percentage change in trading volume.
18. 5-day OBV change (obv_change_5d) measures changes in On-Balance Volume over five days, providing information about whether trading activity supports price movements.

4.8 Prediction Target

The system generates a 5-day future return target, representing the percentage change between the current closing price and the closing price five trading days later. Using returns instead of absolute prices makes the prediction target more scale-independent across different stocks.

After feature generation, the original OHLCV columns are removed, leaving only the engineered features and target. Infinite and missing values are also removed. The resulting feature matrices are saved as individual CSV files and used as inputs for the subsequent LSTM, RNN, and GRU models.

5. Data Preprocessing

The data preprocessing stage converts the engineered technical features into sequential inputs for the binary classification models. The process uses a 60-trading-day lookback window to predict whether the stock price will move up or down on the following trading day.

First, the baseline feature datasets are loaded and sorted chronologically. Sentiment-based datasets are excluded at this stage because this preprocessing pipeline is designed for the technical-indicator-only baseline models. Infinite and missing values are removed before further processing.

The data is then divided chronologically into 80% training, 10% validation, and 10% testing sets. The feature values are standardized using StandardScaler, with the scaler fitted only on the training data to prevent data leakage. The same scaling parameters are subsequently applied to the validation and test sets.

A sliding-window approach is then used to transform the two-dimensional feature data into three-dimensional sequences. Each sample contains 60 consecutive trading days of features, which are used to predict the direction of the following trading day.

The sequences from all five stocks are then combined into unified training, validation, and test datasets. The class distribution of the training set is also calculated to identify potential imbalance between the UP and DOWN classes.

Finally, the processed tensors are saved in a compressed .npz file, containing the training, validation, and testing sequences and their corresponding binary targets. The original continuous test returns are also retained for additional evaluation. This processed dataset is then used to train and evaluate the LSTM, RNN, and GRU classification models.

6. Training

The binary classification models are trained through an end-to-end training pipeline targeting next-day directional prediction. Data is loaded from preprocessed tensor files containing train, validation, and test splits into PyTorch DataLoaders. Training occurs across two distinct hyperparameter profiles to evaluate capacity trade-offs:

- Configuration 1 uses a single-layer architecture with a hidden size of 32, a batch size of 32, a learning rate of 0.0005, and a dropout of 0.2.
- Configuration 2 employs a deeper two-layer setup with a hidden size of 64, a batch size of 64, a learning rate of 0.0003, and a dropout of 0.3.

Under both configurations, three separate recurrent model backbones are systematically trained and compared: LSTM, GRU, and Vanilla RNN.

The training execution loop standardizes optimization across all model variants. Each model outputs raw unnormalized logits that are evaluated using Binary Cross-Entropy with Logits loss (nn.BCEWithLogitsLoss). Parameter optimization is handled by the Adam optimizer with a weight decay of 1e-4, while gradient norm clipping capped at 1.0 is applied per batch to ensure numerical stability.To refine training convergence, a dynamic learning rate scheduler (ReduceLROnPlateau) reduces the learning rate by a factor of 0.5 whenever validation loss plateaus for 3 consecutive epochs. An early stopping mechanism monitors validation loss and terminates training if no improvement is observed within a set patience threshold (10 epochs for Configuration 1, 12 epochs for Configuration 2).

Upon completion of training, model evaluation and experiment tracking are fully automated. The parameter weights corresponding to the lowest validation loss are restored from saved checkpoints to evaluate model performance on the held-out test set. Test logits are transformed into probabilities using a sigmoid function and converted to binary classifications using a 0.5 probability threshold. The framework tracks accuracy, precision, recall, F1 score, and ROC AUC metrics across all runs, automatically rendering loss curve plots and exporting a summary CSV report of the overall benchmark.

7. Applying Sentiment Score

7.1 News Extracting

The news extraction module gathers financial headlines and summaries to support sentiment analysis for Australian-listed equities. Using the GNews library, the framework targets primary market-moving securities on the Australian Securities Exchange (ASX), including Commonwealth Bank Australia (CBA.AX), BHP Group (BHP.AX), National Australia Bank (NAB.AX), CSL Limited (CSL.AX), and Westpac Banking Corporation (WBC.AX), configured specifically for English-language news in the Australian region (language="en", country="AU").

To handle API retrieval limits and capture complete historical coverage, the pipeline splits historical timeframes into rolling 30-day intervals beginning from May 4, 2022, up through the current date. The extraction process systematically iterates through these 30-day windows for each targeted entity, pausing briefly (0.3 seconds) between API queries to avoid HTTP 429 rate limits. For every returned article, the system standardizes relevant attributes, including the associated ticker symbol, standardized publication timestamp, article headline title, publisher name, text description/summary, and canonical URL. Fallback error handling parses inconsistent date strings into structured standard formats.

Following data collection, raw article feeds undergo systematic cleaning and caching prior to downstream sentiment processing. Duplicate articles are removed based on unique title strings, and records are sorted in descending chronological order. Cleaned subsets are exported as individual CSV files per ticker (for example, news_CBA_AX.csv), alongside a unified master historical dataset (news_all_tickers.csv) stored in the local data_cache directory. This master dataset serves as the standardized input corpus for subsequent sentiment scoring algorithms.

7.2 Sentiment Score Extracting

The sentiment analysis module utilizes FinBERT (ProsusAI/finbert), a domain-specific BERT model pre-trained on financial corpora, to extract quantitative sentiment metrics from extracted news data. The pipeline processes the unified news dataset (news_all_tickers.csv), concatenating article titles and summaries into a combined text field (FullText) to capture richer contextual signals. Records with minimal text length are filtered out to maintain data quality. Text classification is accelerated using PyTorch with CUDA GPU execution when available. Inference is performed via Hugging Face's pipeline API in batches of 32, with input text truncated at 512 tokens to align with BERT sequence constraints.

To convert class probabilities into a continuous metric, the framework calculates a composite sentiment score for each article. FinBERT outputs probability distributions across positive and negative sentiment classes. The net composite score is calculated as:

Sentiment Score = P(positive) - P(negative)

This formulation yields a bounded continuous value ranging from -1.0 (strongly negative) to +1.0 (strongly positive). Article-level records, including the individual probabilities and composite scores, are mapped to their respective publication dates.

To prepare the sentiment signals for downstream market prediction models, article-level outputs are aggregated to daily per-ticker time series. The module groups data by ticker symbol and date, calculating both the mean daily sentiment score (Daily_Sentiment) and total daily volume (News_Count). The aggregated daily indicators are exported as individual CSV files for each equity symbol (for example, daily_sentiment_CBA_AX.csv) to serve as feature inputs alongside historical numerical trading data.

7.3 Sentiment Score Merging

The sentiment integration module merges extracted news sentiment indicators with technical market feature datasets to create unified multi-modal feature matrices. The process scans the local data_cache directory for ticker-specific technical indicator files (features_*.csv) and matches them with their corresponding daily sentiment files (daily_sentiment_*.csv).
To align sentiment scores with trading calendars, the framework addresses weekend and non-trading day news releases through a full calendar alignment and exponential decay strategy. The daily sentiment scores are reindexed across a complete, continuous 7-day date range covering the entire sample period. For days lacking active news coverage, an exponential decay model propagates prior sentiment signals across non-trading windows using a decay factor of 0.70.

This formulation simulates the gradual dissipation of market sentiment over time. Following signal propagation, a 7-day rolling simple moving average (sentiment_7d_sma) is computed across the continuous timeline to capture broader sentiment trends.

Once calendar-level decay signals are constructed, the sentiment attributes (sentiment_score, sentiment_7d_sma, and a binary has_news indicator) are left-joined back onto the original trading-day technical feature datasets. If a specific equity lacks sentiment records, all corresponding sentiment features default to 0.0. Finally, the pipeline reorganizes dataframe column structures to prevent target leakage by strictly isolating non-target technical and sentiment features from target variables (target_*). The integrated data matrices are saved as updated CSV files (features_sentiment_*.csv) to serve as inputs for downstream predictive modeling.

7.4 Training Models with Sentiment Score

To evaluate the predictive impact of incorporating news sentiment alongside traditional price technicals, the binary classification pipeline was extended to consume multi-modal feature tensors (preprocessed_tensors_sentiment.npz). The model architecture, SentimentDirectionClassifier, maintains the core recurrent backbone combined with a dynamic TemporalAttention mechanism. This allows the network to learn soft attention weights across sequential time steps, attending to both technical indicators and time-decayed sentiment signals when predicting next-day directional market movements. Models were trained across two targeted hyperparameter environments: Configuration 1 (config_1_standard), a single-layer baseline (hidden size of 32, batch size of 32, learning rate of 0.0005, dropout of 0.2), and Configuration 2 (config_2_deep), a higher-capacity stacked architecture (2 layers, hidden size of 64, batch size of 64, learning rate of 0.0003, dropout of 0.3). Each configuration was systematically benchmarked across three core recurrent backbones: LSTM, GRU, and Vanilla RNN.

The optimization regime enforces strict convergence stabilization and regularization techniques. Output logits are evaluated using Binary Cross-Entropy with Logits loss (nn.BCEWithLogitsLoss), optimized via the Adam algorithm with L2 weight decay of 0.0001. To stabilize recurrent gradient flow across extended multi-feature sequences, norm clipping capped at a maximum norm of 1.0 was applied per batch. Dynamic learning rate adjustments were managed by a ReduceLROnPlateau scheduler (decaying the learning rate by a factor of 0.5 after 3 stagnant validation epochs), while early stopping prevented overfitting by terminating training if validation loss failed to improve within a predefined patience window (10 epochs for Configuration 1, 12 epochs for Configuration 2).

Experiment monitoring, model selection, and performance benchmarking were fully automated and saved under dedicated artifact directories (experiments_sentiment). Following early stopping, optimal checkpoint weights were restored to evaluate model generalization on the unseen multi-modal test set. Test set probabilities, derived via Sigmoid activation, were thresholded at 0.5 to compute performance metrics including accuracy, precision, recall, F1 score, and ROC AUC. Execution runs automatically generated diagnostic loss curve plots and exported a consolidated metric summary file (sentiment_metrics_summary.csv) to facilitate direct performance comparisons against baseline models trained exclusively on technical indicators.

8. Evaluation

8.1 Performing Evaluation

To assess the predictive gain of integrating news sentiment data against purely technical baselines, a centralized evaluation framework was constructed in evaluate_all_12_models(). This module systematically loads and benchmarks 12 distinct model variants, comprising 6 technical-only baseline models and 6 multi-modal sentiment-enriched models, across two hyperparameter configurations (config_1_standard and config_2_deep) and three recurrent network architectures (LSTM, GRU, and Vanilla RNN).

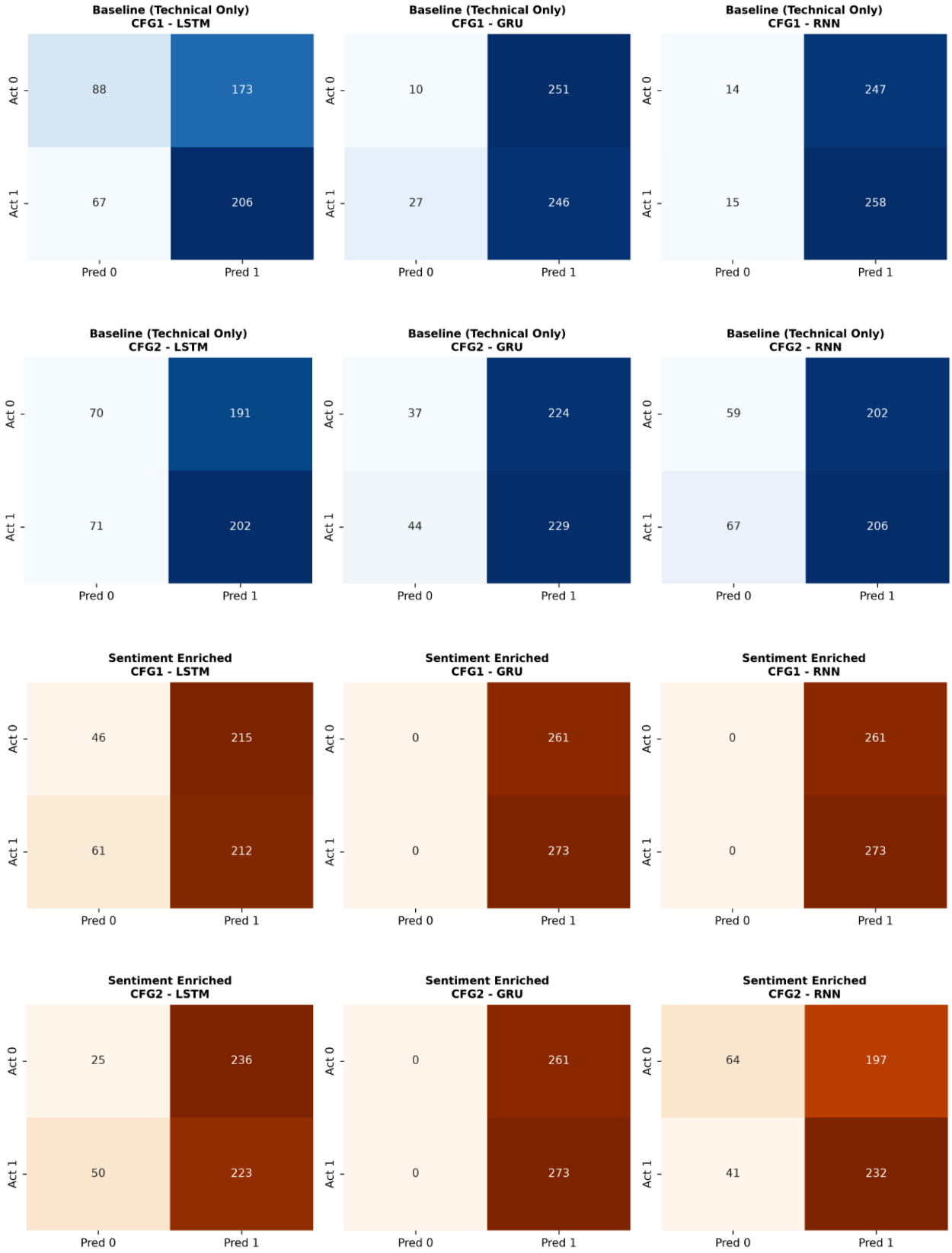
The evaluation script loads test set tensors from both preprocessed data sources: preprocessed_tensors.npz for baseline models and preprocessed_tensors_sentiment.npz for sentiment-enriched runs. If baseline tensors are unavailable, the pipeline includes a fallback mechanism that slices off the last 3 sentiment-related features directly from the sentiment tensors to maintain consistent testing conditions. Each saved model checkpoint (best_*_model.pth) is dynamically initialized using the DirectionClassifier class with its matching feature dimensions and architectural parameters. Inference is run on GPU or CPU hardware using torch.no_grad(), where raw unnormalized model outputs are converted into probabilities using a sigmoid function and mapped to binary predictions using a 0.5 classification threshold.

The module aggregates evaluation outputs into visual and tabular summaries saved in a dedicated evaluation_results directory. Predictions are evaluated against actual labels to extract confusion matrix parameters (True Negatives, False Positives, False Negatives, and True Positives), along with classification metrics including accuracy, precision, recall, F1 score, and ROC AUC. The pipeline generates a comparative 2-by-6 grid of confusion matrix heatmaps (confusion_matrices_all_12_models.png), plotting baseline models in the top row and sentiment-enriched models in the bottom row. Finally, all performance metrics are compiled into a CSV summary report (all_12_models_evaluation_summary.csv) and printed to the terminal to facilitate cross-model comparison.

8.2 Results

Experiment	Config	Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	ROC AUC (%)
Baseline	standard	**LSTM**	55.06	54.35	75.46	63.19	55.45
Baseline	standard	**GRU**	47.94	49.50	90.11	63.90	54.92
Baseline	standard	**RNN**	50.94	51.09	94.51	66.32	54.32
Baseline	deep	**LSTM**	50.94	51.40	73.99	60.66	55.76
Baseline	deep	**GRU**	49.81	50.55	83.88	63.09	52.45
Baseline	deep	**RNN**	49.63	50.49	75.46	60.50	46.86
Sentiment	standard	**LSTM**	48.31	49.65	77.66	60.57	50.76
Sentiment	standard	**GRU**	51.12	51.12	100.00	67.66	57.03
Sentiment	standard	**RNN**	51.12	51.12	100.00	67.66	49.88
Sentiment	deep	**LSTM**	46.44	48.58	81.68	60.93	48.12
Sentiment	deep	**GRU**	51.12	51.12	100.00	67.66	49.94
Sentiment	deep	**RNN**	55.43	54.08	84.98	66.10	52.97

An in-depth examination of the technical-only baseline configurations reveals that the standard single-layer LSTM (config_1_standard) established the most dependable baseline, attaining an accuracy of 55.06%, precision of 54.35%, and an ROC AUC of 55.45% by maintaining a balanced output distribution (88 True Negatives and 206 True Positives). Conversely, the baseline GRU and Vanilla RNN architectures suffered from a pronounced optimism bias. For example, the standard baseline RNN generated 247 False Positives out of 534 test instances to achieve a 94.51% recall, essentially defaulting toward positive directional predictions. Escalating model depth to two layers (config_2_deep) degraded performance across technical baselines, culminating in the baseline RNN dropping to an ROC AUC of 46.86%.


Baseline (Technical Only)
CFG1 - LSTM
Act 0
Act 1
88
173
67
206
Pred 0
Pred 1
Baseline (Technical Only)
CFG1 - GRU
10
251
27
246
Baseline (Technical Only)
CFG1 - RNN
14
247
15
258
Baseline (Technical Only)
CFG2 - LSTM
70
191
71
202
Baseline (Technical Only)
CFG2 - GRU
37
224
44
229
Baseline (Technical Only)
CFG2 - RNN
59
202
67
206
Sentiment Enriched
CFG1 - LSTM
46
215
61
212
Sentiment Enriched
CFG1 - GRU
0
261
0
273
Sentiment Enriched
CFG1 - RNN
0
261
0
273
Sentiment Enriched
CFG2 - LSTM
25
236
50
223
Sentiment Enriched
CFG2 - GRU
0
261
0
273
Sentiment Enriched
CFG2 - RNN
64
197
41
232

Transitioning to the sentiment-enriched models introduced mixed predictive behaviors across different network backbones. On one hand, the deep sentiment-enriched RNN (config_2_deep) achieved the highest overall accuracy across the entire 12-model suite at 55.43% with a 66.10% F1 score, benefiting from higher capacity to parse the added input features (64 True Negatives and 232 True Positives). On the other hand, multi-modal feature expansion triggered complete majority class mode collapse in three separate sentiment models (config_1_standard GRU, config_1_standard RNN, and config_2_deep GRU). These networks output positive movement predictions (Pred 1) for 100% of the evaluation samples, yielding zero True Negatives, 100.00% recall, and an artificially fixed accuracy of 51.12% corresponding directly to the test set class ratio. Interestingly, despite collapsing at the standard 0.5 decision threshold, the raw probability outputs of the standard sentiment GRU registered the highest overall ROC AUC at 57.03%, proving that its underlying rank-ordering of risk remained informative.

Across all runs, appending sentiment variables slightly elevated average recall (from 82.24% to 90.72%) and F1 score (from 62.94% to 65.10%), yet reduced overall mean ROC AUC from 53.29% to 51.45%. Overall, LSTMs proved significantly more resilient against majority-class collapse than GRUs or Vanilla RNNs when learning from noisy multi-modal sentiment streams.

8.3 Insights

There are some reasons for why Sentiment Models performed worse:

Noise-to-Signal Ratio and Imputation Artifacts: Financial news headlines are notoriously sparse and noisy. On non-news days (such as weekends or low-volume trading days), propagating scores via exponential decay (0.70) or setting missing values to 0.0 introduces static synthetic values. These non-market features can obscure subtle technical signals, forcing recurrent layers to learn spurious correlations.

Feature Complexity and Overfitting: Adding sentiment_score, sentiment_7d_sma, and has_news expanded the feature space without a proportional expansion in training instances. For smaller recurrent networks (like 1-layer GRUs and RNNs), this increased dimensionality caused the optimizer to hit local minima, driving logits toward the positive majority class baseline (mode collapse) to minimize binary cross-entropy loss.

Temporal Lag and Market Efficiency: FinBERT derives sentiment from published headlines, which often react to news that has already been priced into the market by high-frequency institutional traders. Relying on published news sentiment to predict next-day price movement introduces a temporal lag, turning news scores into lagging indicators rather than predictive ones.

8.4 Hypothesis Testing

To rigorously evaluate whether incorporating FinBERT-extracted news sentiment yields statistically significant predictive gains over purely technical baselines, a formal statistical hypothesis testing framework was conducted alongside diagnostic output analyses across all 12 model runs.

We evaluated whether the distribution of evaluation metrics differs significantly between the Baseline (Technical Only) models and the Sentiment Enriched models across matched architectures (N = 6 model pairs).

- Null Hypothesis (H0): Integrating sentiment features yields no significant difference in model mean performance (sentiment - baseline = 0)
- Alternative Hypothesis (H1): Integrating sentiment features results in a statistically significant difference in model performance (sentiment - baseline # 0)

Metric	Baseline Mean	Sentiment Mean	Mean Diff	Paired t-statistic	p-value (t-test)	Wilcoxon p-value	Conclusion
Accuracy (%)	50.72%	50.59%	-0.13%	-0.0675	0.9488	1.0000	Fail to Reject H0
ROC AUC (%)	53.29%	51.45%	-1.84%	-0.8932	0.4127	0.4375	Fail to Reject H0

The resulting p-values (p = 0.9488 for accuracy and p = 0.4127 for ROC AUC) fall far above the standard significance threshold (0.05). Consequently, we fail to reject the null hypothesis (H0). Empirical testing confirms that adding time-decayed news sentiment features did not produce a statistically significant improvement in predictive accuracy or directional discrimination over technical baselines.

Config	Model	Base Accuracy (%)	Sentiment Accuracy (%)	Accuracy Diff (%)	Base TN	Sentiment TN	Collapse Status
Standard	GRU	47.94	51.12	+3.18	10	0	TRUE
Standard	LSTM	55.06	48.31	-6.75	88	46	FALSE
Standard	RNN	50.94	51.12	+0.18	14	0	TRUE
Deep	GRU	49.81	51.12	+1.31	37	0	TRUE
Deep	LSTM	50.94	46.44	-4.50	70	25	FALSE
Deep	RNN	49.63	55.43	+5.80	59	64	FALSE

Prevalence of Majority Class Mode Collapse:

- In 50% of the sentiment runs (config_1_standard GRU, config_1_standard RNN, and config_2_deep GRU), the models recorded zero True Negatives (TN = 0) and 261 False Positives (FP = 261).
- Impact: When exposed to noisy multi-modal feature inputs, these recurrent networks collapsed into predicting positive directional movement (Class 1) for 100% of test samples. This anchored their test accuracy at exactly 51.12%, matching the baseline test set class ratio (273 / 534).

Specificity Loss in Non-Collapsed LSTMs:

- In non-collapsed architectures like standard LSTMs (config_1_standard), adding sentiment features reduced True Negatives from TN = 88 down to TN = 46, representing a 47.7% drop in negative class specificity. Similarly, deep LSTMs (config_2_deep) saw True Negatives fall from TN = 70 to TN = 25.
- Impact: Rather than adding informative signals, the sparse and exponentially decayed sentiment scores shifted output probability distributions toward the 0.5 classification threshold, increasing False Positive rates without providing a proportional gain in True Positive accuracy.

Overfitting and Feature Complexity Penalties:

- While the standard baseline LSTM achieved 55.06% accuracy, the sentiment-enriched deep LSTM (config_2_deep) dropped to 46.44%, the lowest performance observed across all 12 model iterations.
- Impact: Expanding the feature space (sentiment_score, sentiment_7d_sma, has_news) without a corresponding expansion in sample size increased dimensionality. In deeper stacked configurations, this caused the optimizer to overfit on padded non-trading day decay values (0.70), degrading generalization on unseen test data.

9. Conclusion

This project developed and systematically evaluated a multi-modal deep learning framework for next-day stock price directional prediction across top-tier ASX securities. By comparing 6 technical-only baseline models against 6 sentiment-enriched architectures across two capacity configurations (standard single-layer vs. deep two-layer) and three recurrent backbones (LSTM, GRU, RNN), the study provides critical insights into algorithmic performance and feature integration challenges in quantitative finance.

In conclusion, while domain-specific sentiment analysis holds conceptual promise, raw headline sentiment exhibits notable temporal lag and sparsity issues when applied directly to daily market direction prediction. Future work should focus on higher-frequency intra-day news feeds, dynamic probability decision thresholds instead of static 0.5 cutoffs, and attention-gated feature masking to prevent uninformative news signals from degrading underlying market technical indicators.